M I N I P R O J E K T

TicketHub

Online Eventbilletsystem

Microservices med C#, Dapr og Aspire

Varighed: 5–7 uger

Teknologier: .NET / C#, Dapr, .NET Aspire

Op til 5 microservices med messaging, REST og SAGA

1. Introduktion

I dette miniprojekt skal I designe og implementere et online eventbilletsystem kaldet **TicketHub**. Systemet gør det muligt for arrangører at oprette events (koncerter, konferencer, teater), og for kunder at søge, bestille og betale billetter.

Projektet strækker sig over 5–7 uger og fører jer gennem hele processen: fra domæneanalyse med Bounded Contexts, over design af kommunikationsmønstre, til implementering af op til 5 microservices med C#, Dapr og .NET Aspire.

1.1 Læringsmål

- Analysere et domæne med Bounded Context-metoden og identificere microservices
- Implementere microservices i C# med .NET Aspire som orkestreringsværktøj
- Anvende Dapr til pub/sub-messaging og service invocation
- Implementere en SAGA med kompenserende transaktioner via Dapr Workflow
- Træffe og begrunde valg mellem asynkron messaging og synkrone REST-kald
- Observere og fejlfinde distribuerede systemer via Aspire Dashboard
- Strukturere en microservice internt efter Clean Architecture med adskillelse af Domain, Use Case, Facade og Infrastructure

1.2 Teknologistak

.NET / C# – Alle services bygges som ASP.NET Core-projekter.

Dapr – Anvendes til pub/sub (messaging), service invocation (REST mellem services) og workflows (SAGA).

.NET Aspire – Orkestreringsværktøj der starter alle services, Dapr sidecars og infrastruktur (f.eks. Redis, RabbitMQ) samlet. Giver et dashboard til at observere logs, traces og metrics på tværs af services.

2. Casebeskrivelse

TicketHub er en platform med følgende overordnede funktionalitet:

- Arrangører kan oprette events med navn, dato, sted, beskrivelse og billetkategorier (f.eks. “Standard”, “VIP”) med hvert sit antal og pris.
- Kunder kan browse og søge i tilgængelige events.
- Kunder kan bestille billetter. En bestilling involverer reservation af billetter, betaling og bekræftelse.
- Hvis betaling fejler eller timer ud, skal reserverede billetter frigives automatisk (kompenserende transaktion).
- Kunder modtager notifikationer ved bekræftelse og annullering.

> **Forretningsregel: Billetter er en begrænset ressource**
> Hver billetkategori har et fast antal billetter. Når en kunde starter en bestilling, skal billetterne reserveres midlertidigt, så de ikke kan sælges til andre. Hvis bestillingen ikke gennemføres, skal reservationen ophæves. Dette er kernemotivationen for SAGA-mønstret.

3. Designniveauer

Arkitekturen i TicketHub kan beskrives på tre niveauer, som tilsammen dækker hele designet fra det store billede ned til den enkelte service:

Overordnet design (afsnit 3.1) handler om at opdele domænet i selvstændige services. Her bruger I Bounded Context-analysen til at identificere hvilke microservices systemet består af, og hvad hver service ejer og er ansvarlig for.

Eksternt design (afsnit 4–5) handler om kommunikationen mellem services. Her beslutter I om en given interaktion skal være synkron (REST via Dapr service invocation) eller asynkron (pub/sub via Dapr), og I designer SAGA-flowet der orkestrerer bookingprocessen på tværs af services.

Internt design (afsnit 6) handler om den interne opbygning af hver enkelt service. Her følger I Clean Architecture-principperne, så hver service er struktureret i lag (Domain, Use Case, Facade, Infrastructure) med tydelig adskillelse af forretningslogik og tekniske detaljer.

Disse tre niveauer hænger tæt sammen: Det overordnede design bestemmer hvilke services der findes. Det eksterne design bestemmer hvordan de taler sammen. Og det interne design bestemmer hvordan hver service er bygget op indeni.

3.1 Overordnet design: Domæneanalyse med Bounded Contexts

Før I begynder at kode, skal I analysere domænet og identificere de Bounded Contexts der skal blive til microservices. Start med at genlæse casebeskrivelsen i afsnit 2 og læg mærke til de forskellige ansvarsområder: Hvem ejer stamdata om events? Hvem holder styr på hvor mange billetter der er tilbage? Hvem koordinerer en bestilling fra start til slut? Hvem håndterer betaling? Og hvem sender beskeder til kunden?

Et godt udgangspunkt er at kigge efter substantiver og handlinger i casebeskrivelsen. Når I finder et område der har sin egen data, sine egne regler og kunne fungere uafhængigt, er det sandsynligvis en selvstændig Bounded Context. Tænk også over: Hvis to begreber lyder ens (f.eks. “billetkategori”), men betyder noget forskelligt alt efter konteksten – så er det et stærkt hint om at de hører til i hver sin context.

Nedenfor er analysen beskrevet – I forventes at gennemarbejde den og dokumentere jeres forståelse.

3.1.1 Identificerede Bounded Contexts

Domænet kan opdeles i følgende fem Bounded Contexts, der hver repræsenterer et selvstændigt ansvarsområde:

Bounded Context	Ansvar
Event Catalog	Arrangører opretter og vedligeholder events. Ejer stamdata: navn, dato, sted, beskrivelse, billetkategorier og priser. Udstiller data til browsing og søgning.
Inventory	Administrerer lagerbeholdning af billetter per event og kategori. Håndterer reservation (hold) og frigivelse (release). Ejer det aktuelle antal tilgængelige billetter.
Ordering	Modtager bestillinger fra kunder. Orkestrerer hele bookingflowet via en SAGA. Ejer ordrelivscyklussen: Created → Confirmed → Cancelled.
Payment	Simulerer en betalingsgateway. Kan lykkes, fejle eller timeoute. Isoleret fordi betaling er et selvstændigt domæne med egne regler og integrationer.
Notification	Sender bekræftelser og annulleringer til kunder. Reagerer på events fra andre services. Har ingen forretningskritisk logik.

Denne opdeling er ikke tilfældig. Ifølge Conway's Law afspejler et systems arkitektur den organisationsstruktur der bygger det. Vendt på hovedet – som "Inverse Conway Maneuver" – kan man bevidst designe sine services efter de ansvarsområder man ønsker adskilt. Hvis ét team ejer billetbeholdningen og et andet ejer betalingslogikken, så bør de også være separate services. Hver af de fem Bounded Contexts ovenfor svarer til et selvstændigt ansvarsområde der kan ejes af ét team.

En god teknik til at synliggøre opdelingen er BPMN swimlane-diagrammer. I et swimlane-diagram får hver aktør (eller service) sin egen vandrette bane, og processens aktiviteter tegnes i den bane der ejer dem. Når en pil krydser fra én bane til en anden, repræsenterer det kommunikation mellem services. Det anbefales derfor at I laver et BPMN swimlane-diagram. Dette er også velegnet til at kommunikerer med jeres Product Owner omkring opgaven.

3.1.2 Hvorfor disse opdelinger?

Event Catalog vs. Inventory: Selvom begge handler om events, er deres ansvarsområder fundamentalt forskellige. Event Catalog ejer stamdata (hvad eksisterer), mens Inventory ejer tilgængelighed (hvor mange er der). Inventory har kompleks logik omkring hold/release og concurrency, som ikke hører hjemme i et katalog.

Ordering som orkestrator: Ordering-servicen ejer SAGA'en og koordinerer de andre services. Den ved hvordan et bookingflow ser ud, men uddelegerer det faktiske arbejde (reservation, betaling) til specialiserede services.

Payment som selvstændig service: I et produktionssystem ville Payment integrere med eksterne betalingsudbydere (Stripe, Nets). Det er et separat domæne med egne fejlmønstre,

timeouts og regler. I dette projekt simuleres betalingen, men arkitekturen afspejler den virkelige verden.

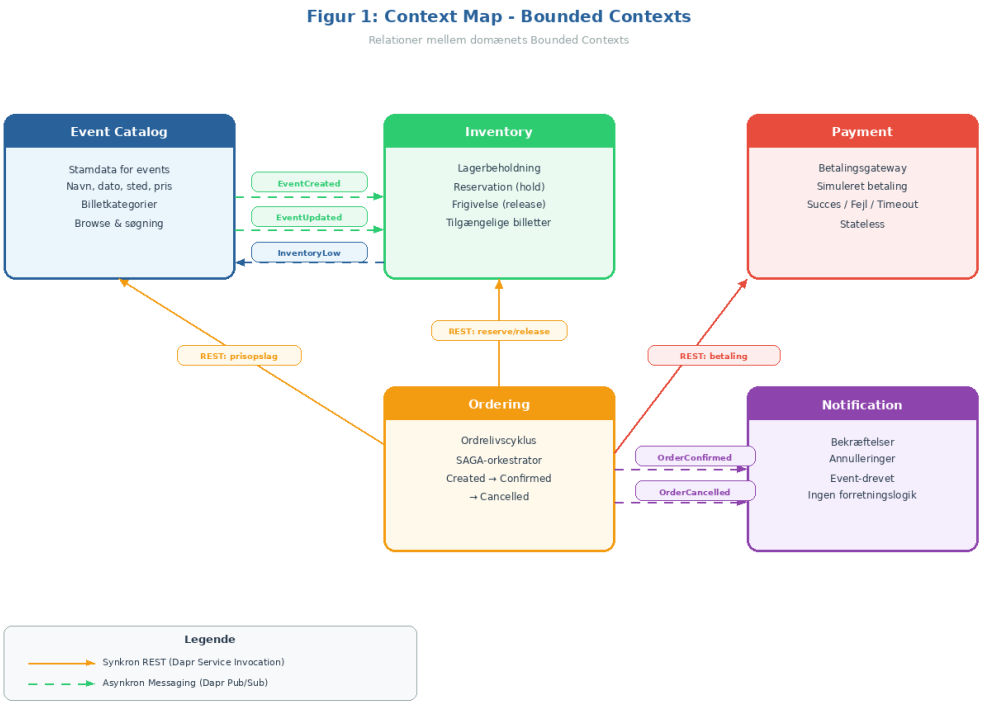
Notification som event-drevet service: Notification har ingen forretningslogik – den lytter passivt på events og reagerer. Det er et klassisk eksempel på en service der udelukkende er koblet via messaging.

3.1.3 Ubiquitous Language

Hver Bounded Context har sit eget sprog. Bemærk at samme begreb kan have forskellig betydning på tværs af contexts:

- I Event Catalog er en “TicketCategory” en definition (navn, pris, total kapacitet).
- I Inventory er en “TicketCategory” en beholdning (antal tilgængelige, antal reserverede).
- I Ordering er “OrderLine” en reference til en billetkategori med et ønsket antal.

Denne forskel i sprogbrug bekræfter at der er tale om separate Bounded Contexts.



Figur 1: Context Map – Relationer mellem Bounded Contexts

4. Eksternt design: Kommunikationsmønstre

Systemet anvender både asynkron messaging (pub/sub) og synkrone REST-kald. Valget er bevidst og begrundet for hvert enkelt tilfælde.

Nøglespørgsmålet er: Skal afsenderen vente på et svar for at kunne gå videre? Hvis ja, er synkron REST det rigtige valg – f.eks. når Ordering har brug for en pris fra Event Catalog, før den kan oprette ordren. Hvis afsenderen derimod bare skal meddele at "noget er sket" og ikke behøver et svar, er asynkron pub/sub bedre – f.eks. når en ordre er bekræftet, og Notification blot skal reagere på det.

Tænk også over kobling: Med pub/sub kender afsenderen ikke modtageren, hvilket gør det nemt at tilføje nye lyttere senere. Med REST er der en direkte afhængighed mellem to services. Gå jeres kommunikationer igennem én for én og overvej: Er der tale om en kommando ("gør dette og fortæl mig resultatet") eller en hændelse ("dette er sket")?

4.1 Event Messaging (Pub/Sub via Dapr)

Pub/sub anvendes hvor services skal reagere på hændelser uden at afsenderen behøver et svar. Afsenderen publicerer en event og fortsætter – modtagerne behandler den asynkront.

Event	Publiceret af	Lytter(e)
EventCreated	Event Catalog	Inventory opretter lagerposter
EventUpdated	Event Catalog	Inventory opdaterer kapacitet
OrderConfirmed	Ordering	Notification sender bekræftelse
OrderCancelled	Ordering	Notification sender annullering
InventoryLow	Inventory	Event Catalog markerer "næsten udsolgt"

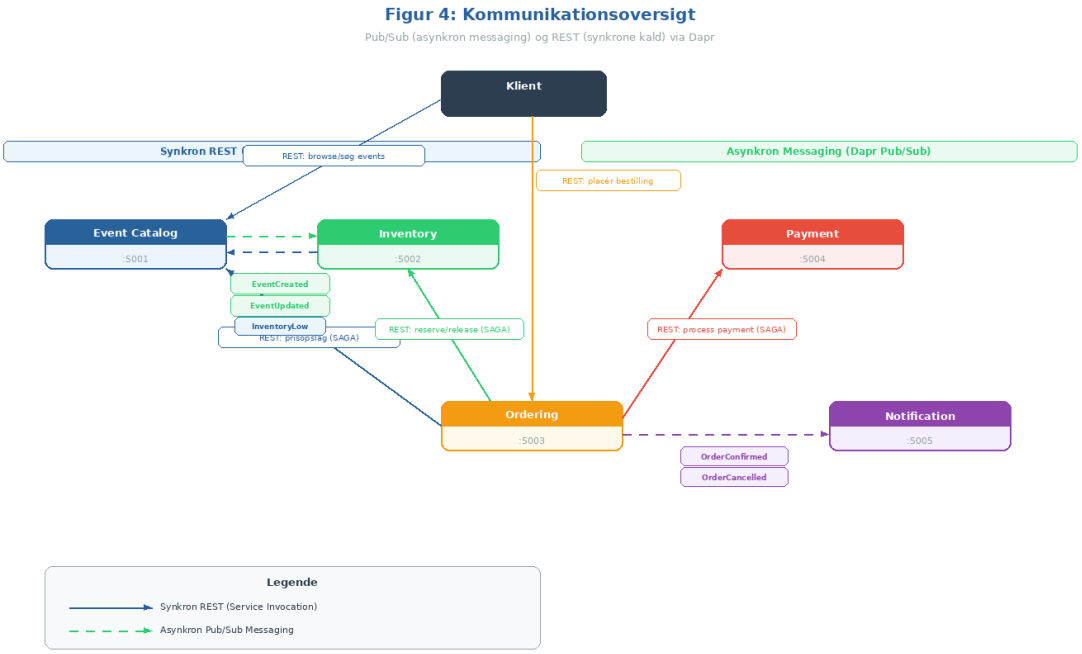
4.2 REST (Synkrone Kald via Dapr Service Invocation)

REST anvendes hvor et svar er nødvendigt med det samme for at kunne fortsætte et flow.

- Kunden browser Event Catalog via REST – det er et klassisk forespørgsel/svar-mønster med ingen sideeffekter.
- Ordering-servicen slår pris op i Event Catalog via Dapr service invocation – prisen skal valideres synkront før ordren oprettes.
- SAGA-workflowet kalder Inventory og Payment synkront via Dapr service invocation – workflowet skal vente på resultatet af hvert trin før det kan beslutte næste handling.

Design-princip

Brug messaging (pub/sub) når afsenderen ikke behøver et svar, og services blot skal reagere på hændelser. Brug REST når et svar er nødvendigt for at fortsætte et flow, eller når data skal læses synkront.



Figur 4: Kommunikationsoversigt – Pub/Sub og REST via Dapr

5. Eksternt design: SAGA – Place Order

Kernen i systemet er bookingflowet, som implementeres som en SAGA med Dapr Workflow i Ordering-servicen. SAGA'en orkestrerer reservation, betaling og bekræftelse – og kompenserer hvis noget fejler.

Hvorfor en SAGA og ikke bare en almindelig transaktion? I et monolitisk system kan man pakke reservation og betaling ind i én database-transaktion. Men i microservices ejer hver service sin egen database, så der findes ingen fælles transaktion. SAGA-mønstret løser dette ved at udføre hvert trin for sig og definere en kompenserende handling, hvis noget går galt undervejs.

Tænk over det konkret: En kunde bestiller 2 VIP-billetter. Først reserveres billetterne i Inventory, så ingen andre kan købe dem. Dernæst forsøges betaling. Men hvad hvis betalingen fejler? Så sidder Inventory med 2 låste billetter, som ingen kan købe. Løsningen er kompensation: Ordering beder Inventory om at frigive billetterne igen. Spørg jer selv ved hvert trin: "Hvad kan gå galt her, og hvordan ruller vi tilbage?"

5.1 Flowbeskrivelse

1. Ordering modtager en bestilling (via REST fra kunden) og opretter en ordre med status Created.
2. Workflowet kalder Inventory for at reservere billetter (ReserveTickets).

✓ Succes: Billetterne er reserveret. Fortsæt til næste trin.

✗ Fejl: Billetterne er ikke tilgængelige. Ordren sættes til Cancelled. Stop.

3. Workflowet kalder Payment for at gennemføre betaling (ProcessPayment).

✓ Succes: Betaling gennemført. Fortsæt til næste trin.

✗ Fejl/Timeout: Betaling fejlede.

→ **Kompensation: Kald Inventory for at frigive billetter (ReleaseTickets).**

→ Ordren sættes til Cancelled. Stop.

4. Bekræft ordren: Sæt status til Confirmed. Publicer OrderConfirmed-event.

→ Notification-servicen lytter og sender bekræftelse til kunden.

5.2 Kompenserende Transaktioner

SAGA-mønstret bruger kompenserende transaktioner i stedet for distribuerede ACID-transaktioner. I dette system er der to kompensationer:

1. ReleaseTickets – Frigiver de reserverede billetter i Inventory, så de igen kan sælges til andre kunder.
2. CancelOrder – Sætter ordrens status til Cancelled og publicerer OrderCancelled-eventen.

5.3 Dapr Workflow-implementering

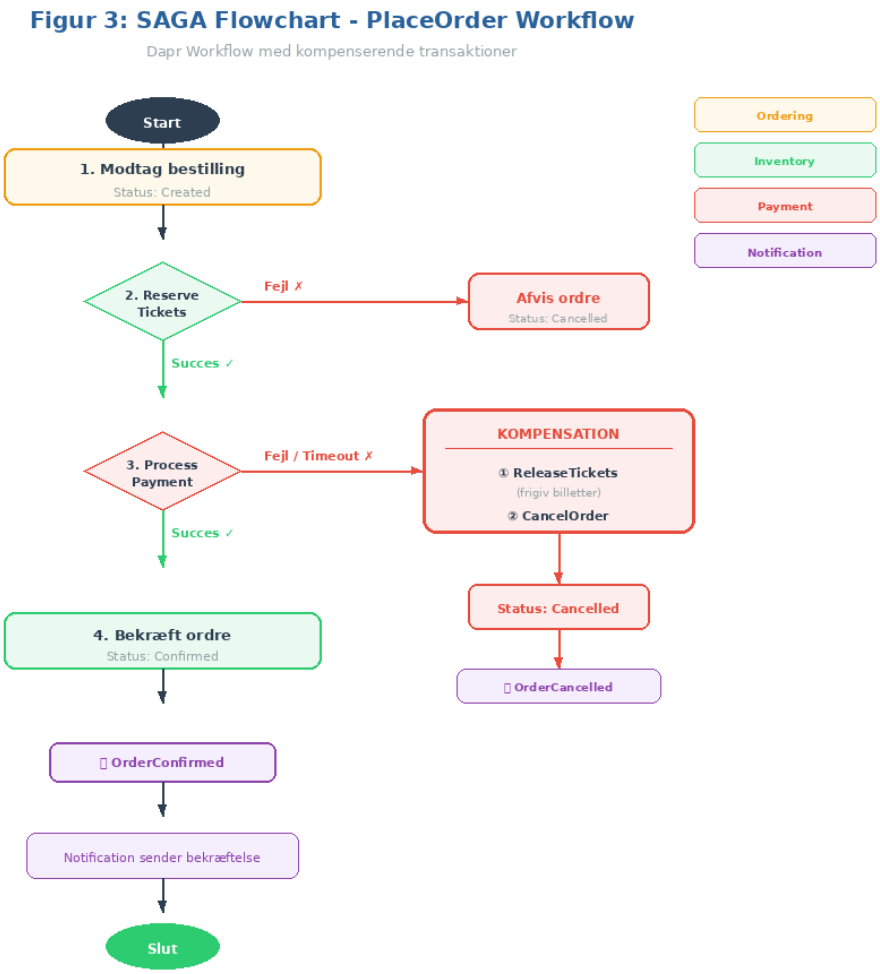
SAGA'en implementeres som et Dapr Workflow i Ordering-servicen. Workflowet defineres som en sekvens af activities, hvor hver activity kalder en ekstern service. Fejlhåndtering og kompensation styres med try/catch i workflowet.

Pseudokode for workflowet:

```
workflow PlaceOrderWorkflow(order):
    try:
        reservationResult = await ReserveTicketsActivity(order)
        if (!reservationResult.success) throw error

        paymentResult = await ProcessPaymentActivity(order)
        if (!paymentResult.success) throw error

        await ConfirmOrderActivity(order)
        publish OrderConfirmed event
        return success

    catch:
        if (tickets were reserved):
            await ReleaseTicketsActivity(order)   // kompensation
        await CancelOrderActivity(order)           // kompensation
        publish OrderCancelled event
        return failure
```

Fejlscenarier I skal teste

1) Billetter ikke tilgængelige – ordren afvises uden kompensation. 2) Betaling fejler – reserverede billetter frigives. 3) Betaling timer ud – som fejl, men drevet af timeout. 4) Alt lykkes – happy path.

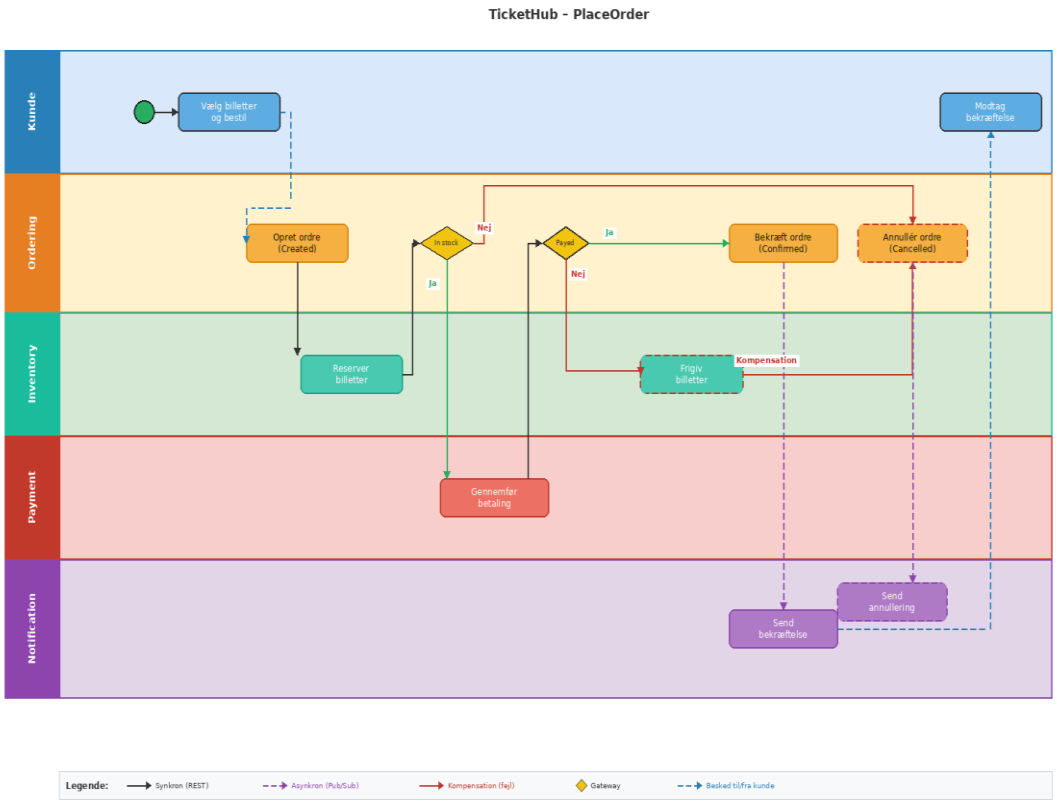


Figur 3: SAGA Flowchart – PlaceOrder Workflow med kompenserende transaktioner

6. Eksternt design: BPMN – Place Order

En god teknik til at synliggøre opdelingen er BPMN swimlane-diagrammer. I et swimlane-diagram får hver aktør (eller service) sin egen vandrette bane, og processens aktiviteter tegnes i den bane der ejer dem. Når en pil krydser fra én bane til en anden, repræsenterer det kommunikation mellem services.

Figur 5 viser bookingflowet som swimlane-diagram. Læg mærke til hvordan hver bane svarer direkte til en Bounded Context fra tabellen ovenfor, og hvordan pilene viser den kommunikation I skal implementere: Sorte pile er synkrone REST-kald, lilla stiplede pile er asynkrone events, og røde pile viser kompensationsflowet ved fejl.



Figur 5: BPMN Swimlane – PlaceOrder-flowet med ansvarsfordeling på tværs af services

Conway's Law i praksis
Læg mærke til at hver swimlane i diagrammet svarer til én Bounded Context fra tabellen ovenfor. Det er Conway's Law i praksis: Systemets struktur følger ansvarsfordelingen. Hvis I forestiller jer at fem forskellige teams skulle bygge TicketHub, ville det være naturligt at lade hvert team eje præcis én swimlane – og dermed én microservice.

6. Internt design: Clean Architecture

Hvor afsnit 3–5 handler om det overordnede og eksterne design (hvilke services findes, og hvordan taler de sammen), handler dette afsnit om det interne design: Hvordan er den enkelte service bygget op indeni?

Hver microservice i TicketHub skal internt følge Clean Architecture-principperne, som kombinerer idéer fra Onion Architecture, Domain-Driven Design (DDD) og Command Query Separation (CQS). Det centrale princip er Dependency Rule: kildekode-afhængigheder må kun pege indad – fra de ydre, tekniske lag mod de indre, forretningsnære lag. Det sikrer at forretningslogikken er uafhængig af frameworks, databaser og andre tekniske detaljer.

6.1 Lagmodellen

Arkitekturen består af fire lag plus et Api-projekt. Hvert lag afspejles som et selvstændigt projekt i jeres .NET Solution:

Domain-laget (inderst) – Arkitekturens hjerte. Indeholder kerneforretningslogikken i sin reneste form: entity-klasser med DDD-principper (private setters, tilstandsændringer via metoder), value objects og domain services. Dette lag har ingen afhængigheder til andre lag eller frameworks overhovedet. I Inventory-servicen vil f.eks. entity-klassen TicketStock med logik for reservation og frigivelse høre til her.

Use Case-laget – Orkestrerer applikationslogikken. Hvert use case repræsenterer én konkret handling (f.eks. “Placér ordre” eller “Reserver billetter”). Use cases kalder Domain-laget for forretningsregler og definerer repository-interfaces (f.eks. IOrderRepository, ITicketStockRepository), som Infrastructure-laget implementerer. Med CQS-princippet er use cases udelukkende commands – de ændrer tilstand, men returnerer ikke data. Læsning af data sker via separate query-interfaces (se Facade-laget).

Facade-laget – Kontraktlaget der udgør den eneste indgang til kernen. Her findes udelukkende interfaces og DTO’er – ingen implementering. Interfaces opdeles efter CQS-princippet: ét interface per use case (command) og ét interface per query-gruppe. Alle parametre og returværdier er DTO’er – domæne-entities eksponeres aldrig. Facade-laget sikrer at API-controllere, test-klienter og andre konsumenter aldrig taler direkte med use cases eller domæne-entities.

Infrastructure-laget (yderst) – Indeholder al teknisk implementering: databaseadgang (Entity Framework, PostgreSQL), Dapr-integrationer, eksterne API-kald og messaging. Infrastructure implementerer både Use Case-lagets repository-interfaces og Facade-lagets query-interfaces. Med CQS kan query-handlers gå direkte til databasen med optimerede forespørgsler uden at involvere domæne-entities.

Api-projektet – ASP.NET Core Web API-controllere der eksponerer servicens funktionalitet. Controllerne refererer kun til Facade-laget (use case-interfaces og query-interfaces) og bruger dependency injection til at resolve de konkrete implementeringer.

6.2 Dependency Rule

Den vigtigste regel i hele arkitekturen er: kildekode-afhængigheder må kun pege indad. Domain-laget kender intet til de andre lag. Use Case-laget afhænger kun af Domain, men implementerer Facade-lagets interfaces. Facade-laget definerer kontrakten (interfaces og

DTO'er) og kender ikke til andre lag. Infrastructure-laget implementerer interfaces fra både Facade og Use Case, og afhænger desuden af Domain (til materialisering og persistering). Api-projektet refererer kun til Facade.

Konkret: Når et Use Case har brug for at gemme data, definerer det et repository-interface (f.eks. IOrderRepository). Infrastructure-laget implementerer dette med Entity Framework og PostgreSQL. Ved opstart registreres implementeringen via dependency injection, så Use Case-laget aldrig har en direkte reference til Infrastructure-projektet. På samme måde definerer Facade-laget query-interfaces (f.eks. IEventQueries), som Infrastructure implementerer med optimerede læseforespørgsler.

6.3 Eksempel: Ordering-servicen

For Ordering-servicen kunne Solution-strukturen se sådan ud: Ordering.Domain indeholder Order-entiteten med statuslogik (Created, Confirmed, Cancelled) og OrderLine som value object. Ordering.UseCases indeholder use cases som PlaceOrderUseCase samt repository-interfaces som IOrderRepository. Ordering.Facade indeholder use case-interfaces (f.eks. IPlaceOrderUseCase), query-interfaces (f.eks. IOrderQueries) og DTO'er (f.eks. OrderDto, PlaceOrderRequest). Ordering.Infrastructure implementerer IOrderRepository med Entity Framework, IOrderQueries med optimerede læseforespørgsler, og Dapr-integrationer. Ordering.Api indeholder controllere der kun refererer til Facade-lagets interfaces.

6.4 Referencemateriale

Følg Clean Architecture-noten fra kursusmaterialet for den fulde beskrivelse af lagene, DDD-entity-mønstret, CQS-princippet og Facade-lagets rolle:

https://github.com/kbr-ucl/DMU-2-Semester-Programmering/blob/main/Noter/Clean-Architecture.md

Krav til internt design
Minimum én service (f.eks. Ordering eller Inventory) skal være struktureret efter Clean Architecture med separate projekter for Domain, UseCases, Facade, Infrastructure og Api. Dependency Rule skal overholdes: Domain må ikke have referencer til andre projekter, Use Cases må kun referere Domain, og Facade må ikke referere nogen andre lag. Api-projektet må kun referere Facade.

7. Ugeplan

Nedenfor er en anbefalet ugeplan. Ugerne 6–7 er valgfrie og kan bruges til udvidelser og forbedringer.

Uge	Fokus	Detaljer
1	Domæneanalyse og setup	Analysér domænet med Bounded Context-metoden. Identificér contexts, definér ubiquitous language per context. Tegn et Context Map der viser relationer mellem contexts. Tegn et BPMN diagram for ordreoprettelsen. Opsæt Aspire-løsning med tomme C#-projekter f.eks. en Event Catalog service, og med fokus på Domain og Facade bliver kodet.
2	Event Catalog og Inventory	Implementér Event Catalog med CRUD-endpoints for events. Implementér Inventory med egen database. Opsæt Dapr pub/sub så Inventory reagerer på EventCreated. Test at lagerposter oprettes automatisk når et event publiceres.
3	Ordering-service	Implementér Ordering med REST-endpoint til at placere en ordre. Tilføj Dapr service invocation mod Event Catalog for prisopslag. Implementér ordrelivscyklus (Created → Confirmed → Cancelled). Test synkron kommunikation mellem Ordering og Event Catalog.
4	Payment og SAGA	Implementér Payment-service med simuleret betaling. Gør fejlrate konfigurerbar (succes, fejl, timeout). Byg SAGA'en som Dapr Workflow i Ordering-servicen. Implementér kompenserende transaktioner (ReleaseTickets, CancelOrder). Test alle fejlscenarier.
5	Notification og integration	Implementér Notification-service der lytter på OrderConfirmed og OrderCancelled. Notifikationer kan simuleres via konsol-output eller logfiler. Test hele flowet end-to-end inkl. fejlscenarier. Observér traces og logs i Aspire Dashboard.
6	Robusthed (valgfri)	Tilføj retry-policies på Dapr-kald. Implementér idempotens på alle endpoints. Opsæt dead letter topics til fejlede beskeder. Tilføj InventoryLow-event fra Inventory til Event Catalog.

7	Udvidelser (valgfri)	Byg et simpelt UI (f.eks. Blazor eller React) til at browse events og bestille. Tilføj et API Gateway foran services. Eksperimenter med skalering af individuelle services. Skriv afsluttende dokumentation.

8. Krav til løsningen

8.1 Minimum (uge 1–5)

3. Dokumenteret domæneanalyse med Bounded Contexts, ubiquitous language og Context Map.
4. Minimum 3 kørende microservices: Event Catalog, Inventory og Ordering.
5. Mindst ét pub/sub-flow (f.eks. EventCreated → Inventory).
6. Mindst ét REST-kald via Dapr service invocation (f.eks. prisopslag).
7. En fungerende SAGA med mindst én kompenserende transaktion implementeret som Dapr Workflow.
8. Aspire-løsning der starter alle services samlet.
9. Minimum én service struktureret efter Clean Architecture med separate projekter for Domain, UseCases, Facade, Infrastructure og Api. Dependency Rule skal overholdes (se afsnit 6).

8.2 Udvidet (uge 6–7)

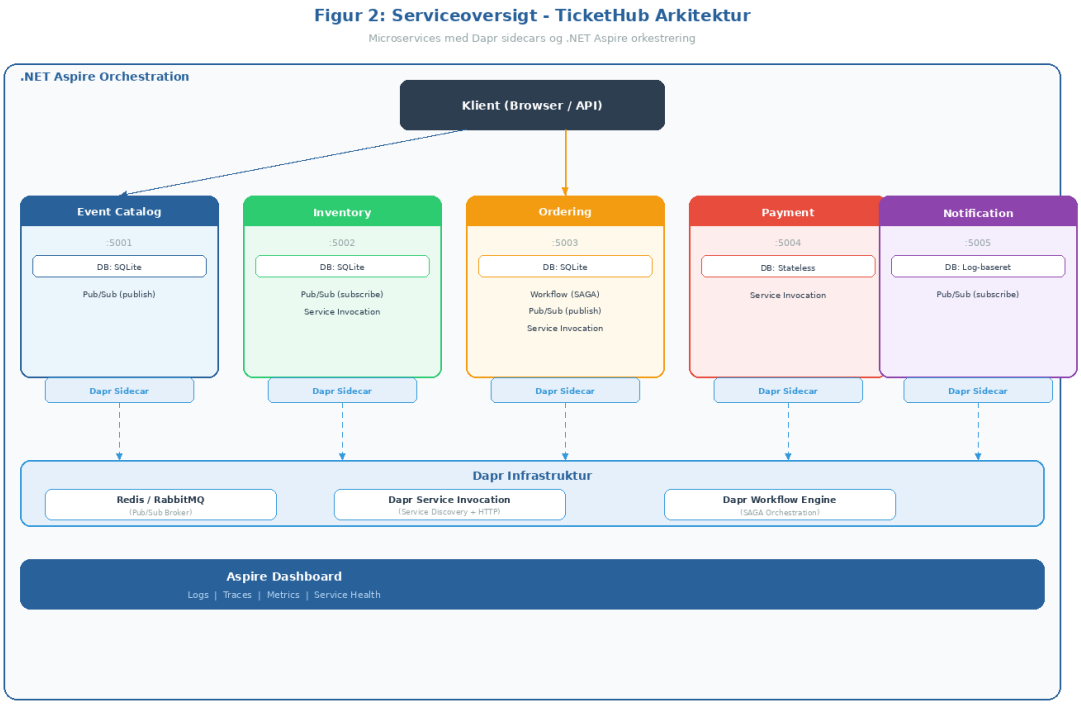
- Payment- og Notification-service (op til 5 services i alt).
- Retry-policies og idempotens.
- Dead letter topics.
- Simpelt UI.
- Afsluttende dokumentation med arkitekturbeskrivelse og refleksion.

9. Vejledende Arkitektur

Nedenfor er en oversigt over den samlede arkitektur. Pilene viser kommunikationsretningen.

9.1 Serviceoversigt

Service	Port (forslag)	Database	Dapr-funktioner
Event Catalog	5001	SQL/PostgreSQL	Pub/sub (publish)
Inventory	5002	SQL/PostgreSQL	Pub/sub (subscribe), Service invocation
Ordering	5003	SQL/PostgreSQL	Workflow, Pub/sub (publish), Service invocation
Payment	5004	Ingen (stateless)	Service invocation
Notification	5005	Ingen (log-baseret)	Pub/sub (subscribe)



Figur 2: Serviceoversigt – TicketHub Arkitektur med Dapr og Aspire

9.2 Kommunikationsoversigt

Asynkron messaging (pub/sub):

- Event Catalog → [EventCreated] → Inventory
- Event Catalog → [EventUpdated] → Inventory
- Ordering → [OrderConfirmed] → Notification

- Ordering → [OrderCancelled] → Notification
- Inventory → [InventoryLow] → Event Catalog

Synkron REST (Dapr service invocation):

- Klient → Event Catalog (browse/søg events)
- Klient → Ordering (placér bestilling)
- Ordering → Event Catalog (prisopslag)
- Ordering (workflow) → Inventory (reserve/release)
- Ordering (workflow) → Payment (process payment)

10. Tips og Gode Råd

Start småt

Begynd med to services og ét kommunikationsmønster. Få det til at virke før I tilføjer mere. Aspire gør det nemt at tilføje services løbende.

Brug Aspire Dashboard aktivt

Dashboard'et viser logs, traces og metrics på tværs af alle services. Det er jeres vigtigste værktøj til at forstå hvad der sker i systemet, især når noget fejler.

Simuler fejl bevidst

Gør Payment-servicens fejlrate konfigurerbar (f.eks. via en miljøvariabel eller et konfigurationsendpoint). Så kan I nemt teste både happy path og fejlscenarier.

Husk idempotens

Messaging-systemer kan levere en besked mere end én gang. Overvej fra starten hvad der sker hvis en handler modtager den samme event to gange. Brug f.eks. et unikt EventId eller OrderId til at sikre idempotens.

Hver service ejer sin data

Services må aldrig dele database. Hvis Ordering har brug for prisdata, kalder den Event Catalog – den læser ikke direkte i Event Catalogs database. Dette princip er fundamentalt for microservices.

Dokumenter jeres valg

Skriv ned hvorfor I valgte pub/sub vs. REST for hver kommunikation. Det er mindst lige så vigtigt at kunne begrunde designet som at implementere det.

— God fornøjelse med projektet! —